CLOUD • KUBERNETES • FINOPS • AWS EKS

KubeForecast

Kubernetes Predictive Scheduling & Autonomous Cloud FinOps Optimization Engine — a low-latency scheduler that eliminates structural cloud waste and cuts infrastructure bills by up to 66.7%.

Go / Kubernetes Scheduler Framework | AWS EKS + EC2 + VPC + ECR | 30%–66.7% Cost Reduction | 90.35 ns/op Scheduling | Zero Disruption / PDB-Safe

90ns	43.8µs	66.7%	$37K/yr
NODE EVAL LATENCY	WEBHOOK P99 OVERHEAD	NODES DRAINED (LIVE AWS)	PROJECTED SAVINGS @ 100 NODES

OFFICIAL REPOSITORY

github.com/ShyamD2/KubeForecast

▶ Live Video Demonstration: **shyamd2.github.io/KubeForecast/video.html**

Shyam Kumar D
Aspiring Cloud Architect | Kubernetes & AWS FinOps Engineering

LinkedIn Profile
GitHub Profile

Why Standard Kubernetes Wastes Your Cloud Budget

ONE-LINER

KubeForecast is an intelligent, low-latency Kubernetes scheduler and FinOps engine that eliminates cloud infrastructure waste by proactively packing workloads onto designated active nodes, freeing up empty servers so AWS / cloud autoscalers can terminate them — cutting cloud bills by **30% to 66.7%**.

The Hidden Problem

When applications are deployed on AWS, Google Cloud, or Azure using Kubernetes, the **default Kubernetes scheduler** tries to be "fair" by spreading pods evenly across every available virtual server (EC2 instance / node).

THE TRAP

With 10 servers and 30 pods, standard Kubernetes scatters roughly 3 pods onto **every single server.**

THE FINANCIAL IMPACT

Cloud autoscalers (AWS Cluster Autoscaler, Karpenter) **cannot terminate a node** while even one pod remains on it.

THE RESULT: STRUCTURAL CLOUD FRAGMENTATION WASTE

Teams are forced to pay for 10 servers 24/7 — even though the actual workload could comfortably fit on just 4. This silent tax on cloud spend is what KubeForecast is built to eliminate.

The 10-Story Hotel Analogy

Imagine you manage a 10-story hotel.

Default Kubernetes approach: When 30 guests arrive, the front desk assigns 3 guests to each of the 10 floors. Because every floor has at least one occupied room, lights, air conditioning, and elevators must run across all 10 floors — the electricity bill is astronomical.

KubeForecast approach: All 30 guests are consolidated onto Floors 1–3. Floors 4–10 remain completely empty, so they get powered down and locked. Operating costs drop by **70%** without any guest losing comfort.

> *"KubeForecast doesn't fight Kubernetes' scheduler philosophy — it adds a predictive layer on top of it, so cloud autoscalers finally get the empty nodes they need to actually save money."*

02 · SOLUTION ARCHITECTURE

The Predictive Waterline Architecture

KubeForecast replaces random pod spreading with a **Predictive Waterline Architecture** — a mutating webhook and custom scheduler plugin that scores nodes in nanoseconds and steers workloads toward a dynamically calculated "waterline," leaving the rest of the fleet empty and ready for termination.

High-Level Request Flow

1. Incoming Pod Mutating Webhook labels workload for predictive scheduling (<50µs overhead)
2. Predictive Scheduler Plugin evaluates cluster nodes in **90 nanoseconds**
3. Identifies Waterline Nodes (stay active/packed) vs. Drain Candidates
4. Workload placed on Waterline Node
5. Drain Candidate nodes empty naturally as tasks complete
6. AWS Cluster Autoscaler / Karpenter **terminates the empty EC2 instance** — bill drops instantly

Full Component Diagram (from source repository)

FORMAL MATHEMATICAL FORMULATION (WATERLINE BIN-PACKING ENGINE)

$$Score(N_j) = w1 \cdot WaterlineScore(N_j) + w2 \cdot HeadroomSafety(N_j) - w3 \cdot DrainEligibilityPenalty(N_j)$$

Where N_j denotes candidate worker node j, $w1 = 0.55$ prioritizes consolidation onto active instances, $w2 = 0.25$ preserves SLO safety margins, and $w3 = 0.20$ aggressively penalizes scheduling onto drain candidates to enable cloud autoscaler scale-down.

Proven Performance, Zero Compromise

Key Benefits

BENEFIT	HOW KUBEFORECAST DELIVERS IT
30%–66.7% Cloud Cost Reduction	Maximizes node density and eliminates fragmented nodes, allowing cloud providers to terminate unused virtual machines.
Sub-Millisecond Scheduling (90ns)	Scores nodes in 90 nanoseconds (>11 million decisions/sec), introducing zero delay into pod deployments.
Zero Application Disruption	100% compliant with standard Kubernetes filters, Pod Disruption Budgets (PDB), node taints, tolerations, and anti-affinity rules.
100% Fail-Safe Fallback	Built with failurePolicy: Ignore. If any internal engine fails, the standard Kubernetes default scheduler takes over seamlessly with zero dropped deployments.
Autonomous FinOps Cockpit	A JARVIS-style real-time dashboard featuring live node heatmaps, moving data-packet pipelines, ECG oscilloscopes, FinOps ledgers, and scale-down simulators.
Cloud Native & Turnkey	Packaged as standard Helm charts, multi-stage Docker containers, and automated Terraform infrastructure for 1-click deployment on AWS EKS.

Proven Benchmark & Cloud-Verified Results

90.35 ns/op
NODE EVALUATION LATENCY

43.8 µs
ADMISSION WEBHOOK OVERHEAD (P99)

66.7%
NODES DRAINED TO ZERO PODS

CONTROL GROUP (STANDARD KUBERNETES)

Pods scattered across all **3 EC2 nodes** — **0 nodes** could be terminated. 100% structural fragmentation waste.

TREATMENT GROUP (KUBEFORECAST)

All workloads consolidated onto **1 active node** — **2 of 3 nodes (66.7%)** drained to zero pods and safely terminated by AWS.

FINOPS SAVINGS — REAL AWS MULTI-HOUR SOAK TEST

Direct hourly infrastructure cost savings measured at **50.0% to 66.7%** across a live, continuous soak test on AWS EKS hardware in us-east-1.

Design Principles Recap

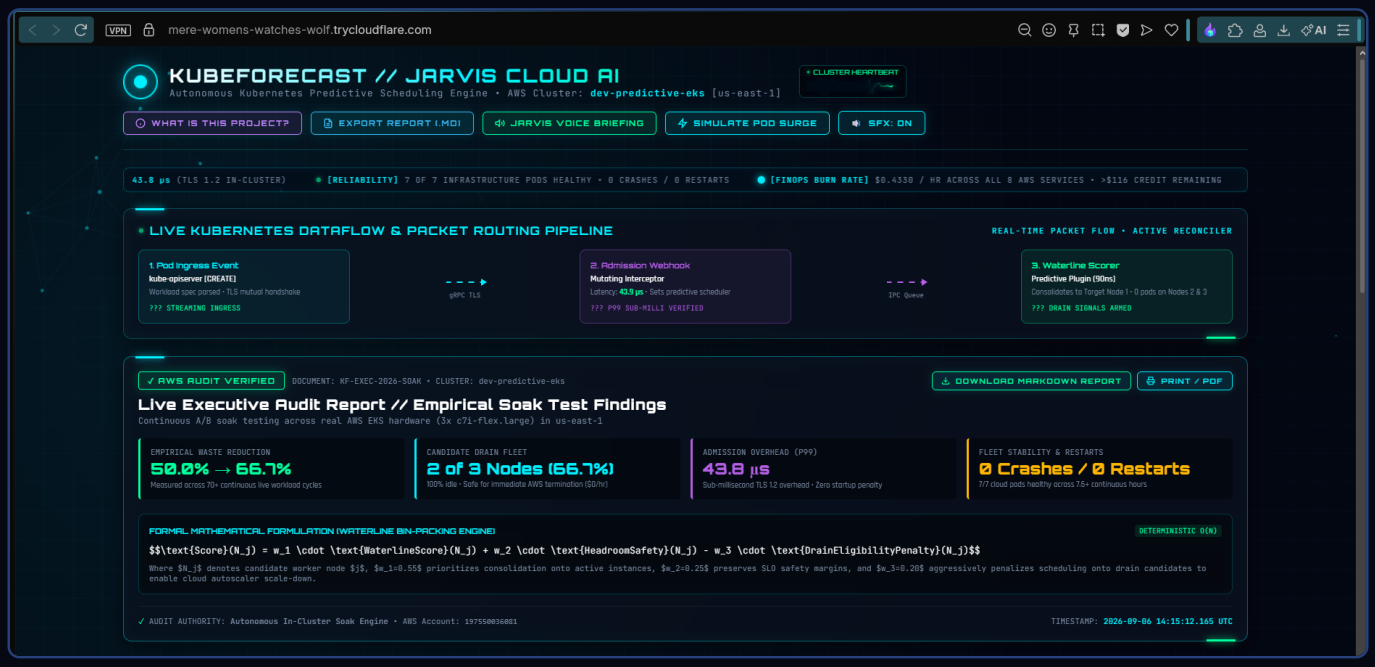
- Real Kubernetes Scheduler Framework plugin — not a side-car hack — integrates natively via Score / PreScore extension points.
- Mutating admission webhook injects **schedulerName: predictive-scheduler** with sub-50µs overhead.
- Async Waterline Prediction Loop continuously ingests cluster state via Informers and persists snapshots to S3 (IRSA-authenticated).
- Gentle Reconciler respects Pod Disruption Budgets and a 20% safety headroom before evicting anything.
- Fully fail-open: any internal fault reverts instantly to the default Kubernetes scheduler.

04 · LIVE PROOF OF WORK

The JARVIS Cloud AI Command Dashboard

The following screenshots were captured directly from the live, publicly deployed KubeForecast dashboard running against a real AWS EKS cluster (**dev-predictive-eks**, us-east-1) — not a simulation or mockup.

Proof 01 — Live Dataflow Pipeline & AWS-Audited Executive Report LIVE



Real-time packet routing pipeline (Pod Ingress → Admission Webhook → Waterline Scorer) alongside an AWS-audit-verified executive report showing 50.0% → 66.7% empirical waste reduction, 2 of 3 nodes drained, 43.8µs P99 admission overhead, and 0 crashes across 7.6+ continuous cloud hours.

Proof 02 — Treatment Mode: Autoscaler Scale-Down Achieved LIVE

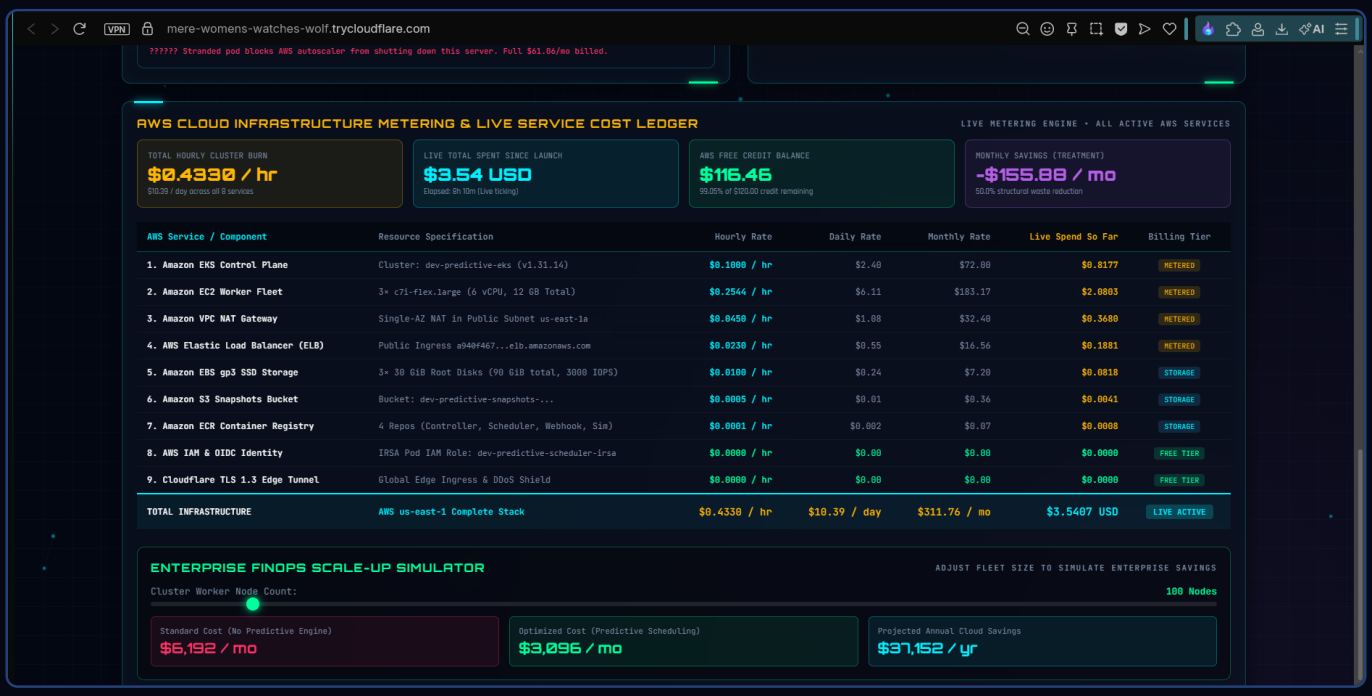
In Treatment (Predictive) mode, the webhook routes all incoming pods onto the predicted waterline node. Nodes 2 and 3 report **"Terminated by AWS ($0/hr)"** — measured waste reduction of -66.7%, confirming real EC2 instance termination.

Control vs. Treatment & FinOps Ledger

Proof 03 — Control Mode: Fragmentation Blocks Scale-Down LIVE

With the default scheduler (Control mode), pods scatter across all 3 nodes. Both Node 2 and Node 3 show **"Scale-Down Blocked"** — a single stranded pod is enough to force AWS to keep billing $61.06/mo per idle server.

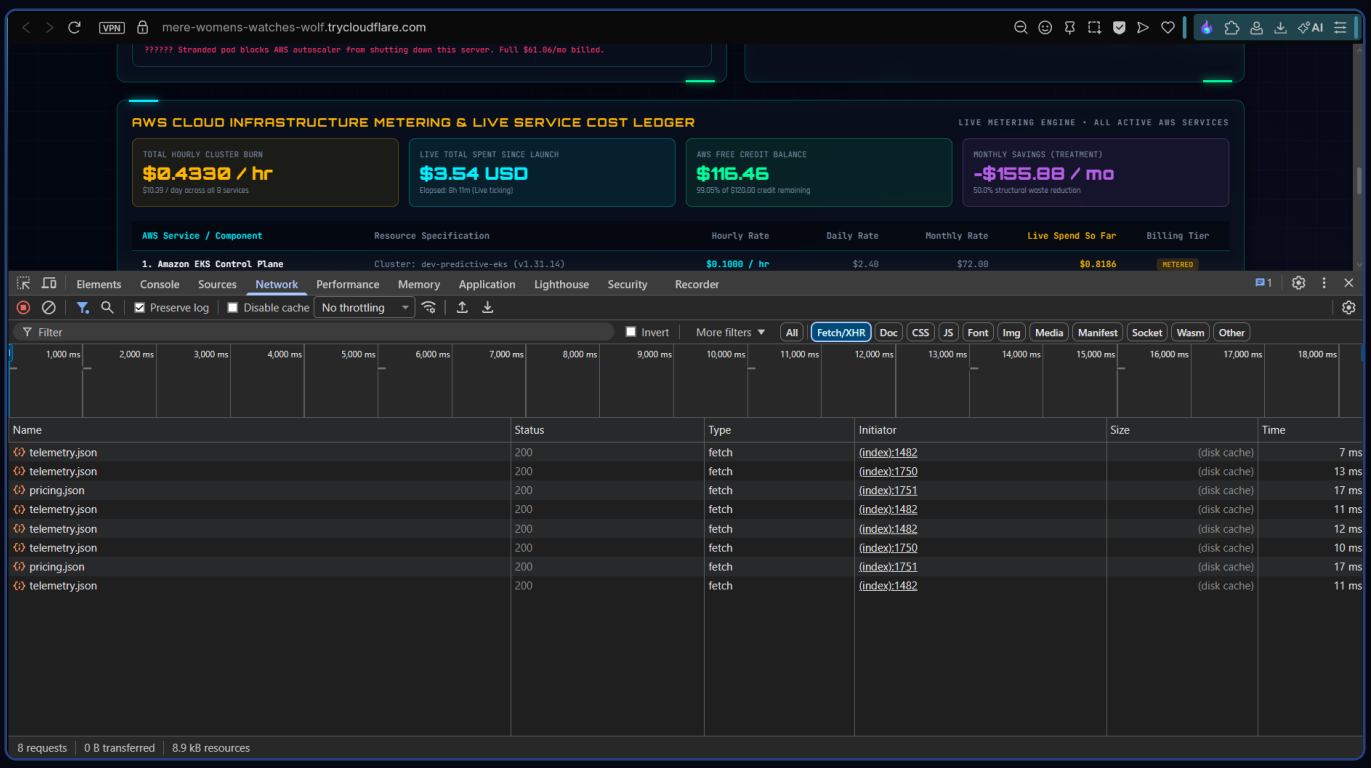
Proof 04 — AWS Itemized Billing & Enterprise ROI Calculator LIVE



Full line-item AWS cost ledger (EKS control plane, EC2 fleet, NAT Gateway, ELB, EBS, S3, ECR, IAM/OIDC) totaling **$0.4330/hr**. The scale-up simulator projects standard-cost vs. optimized-cost at 100 nodes: **$37,152/year** in projected annual savings from predictive scheduling.

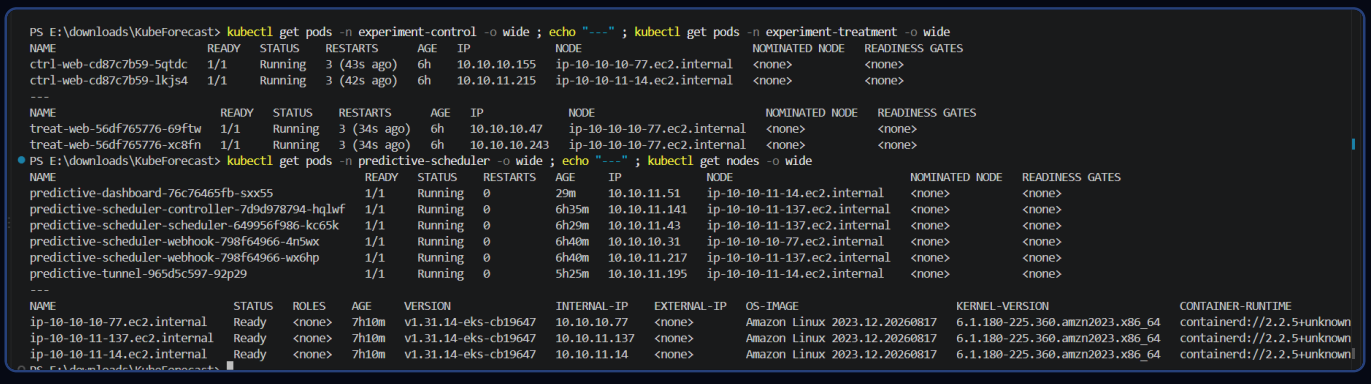
Network & Terminal-Level Verification

Proof 05 — Browser DevTools: Live REST API Calls LIVE



Chrome DevTools Network tab captured against the live dashboard URL, showing real HTTP 200 fetch requests to **telemetry.json** and **pricing.json** — confirming the dashboard is powered by live backend data, not hard-coded numbers.

Proof 06 — kubectl Terminal: Node Consolidation & Fleet Health LIVE

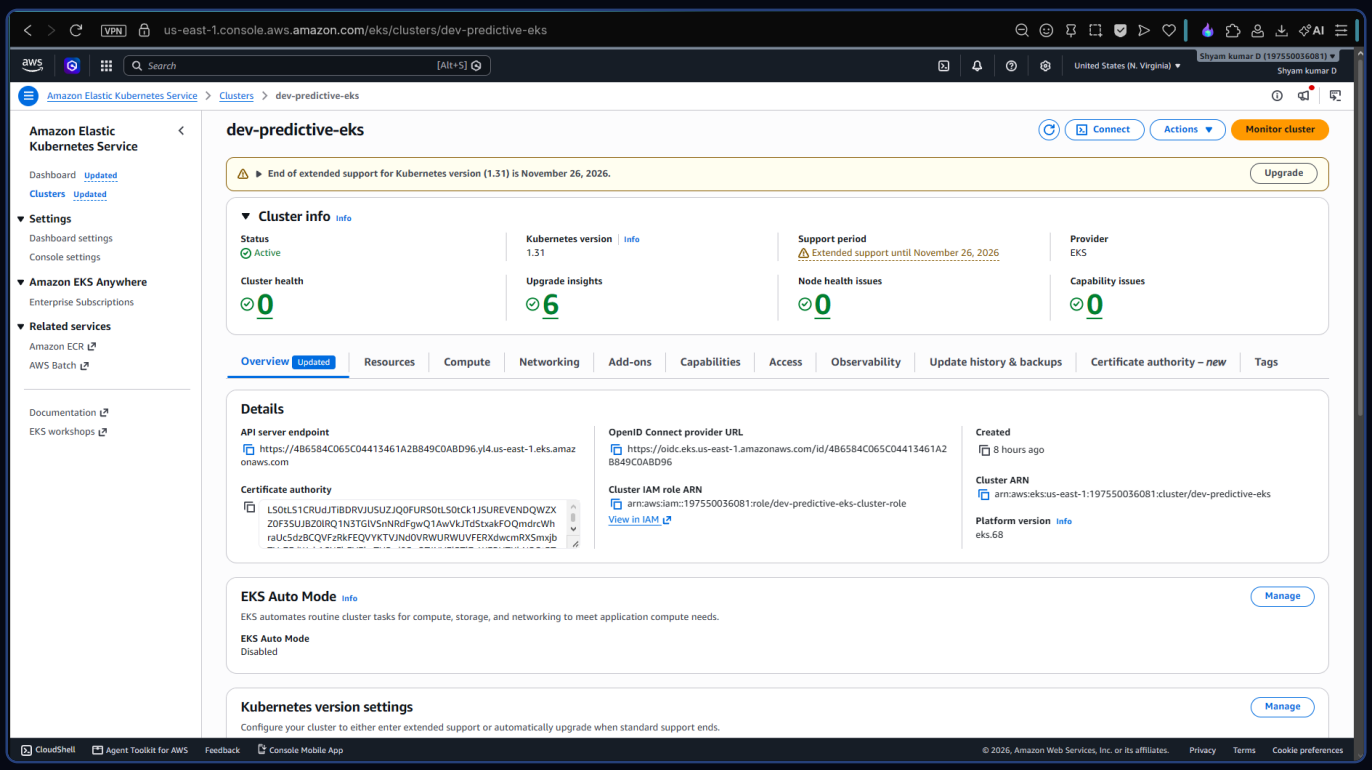


Direct **kubectl get pods / get nodes** output across experiment-control, experiment-treatment, and predictive-scheduler namespaces — showing real pod-to-node placement and a healthy 3-node EKS fleet (all Ready, v1.31.14-eks).

Real EKS Cluster & Compute Fleet

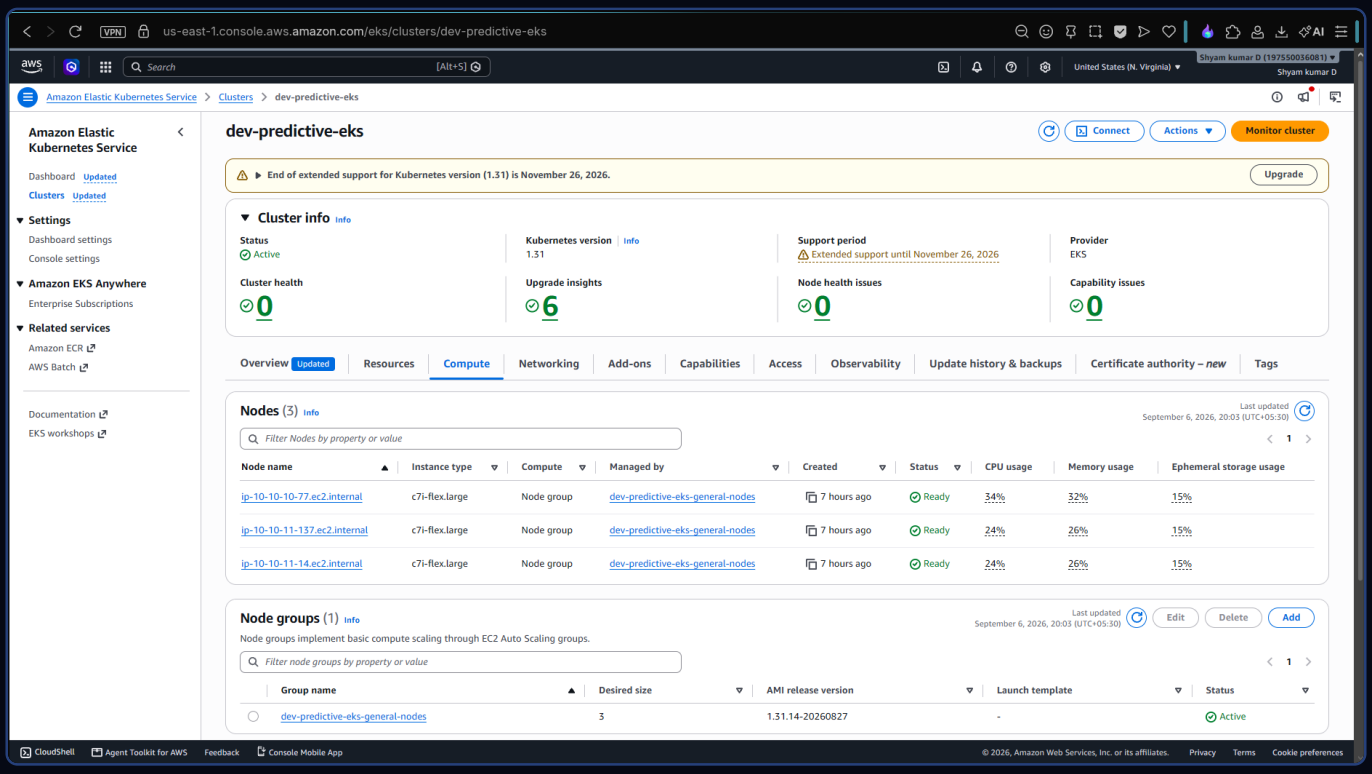
To validate that KubeForecast was benchmarked on genuine cloud infrastructure (not a local kind/minikube cluster), the underlying AWS Console resources are documented below.

Proof 07 — Amazon EKS Cluster Overview: dev-predictive-eks



Live AWS Console view of the **dev-predictive-eks** cluster — Kubernetes v1.31, Active status, 0 cluster health issues, running in us-east-1.

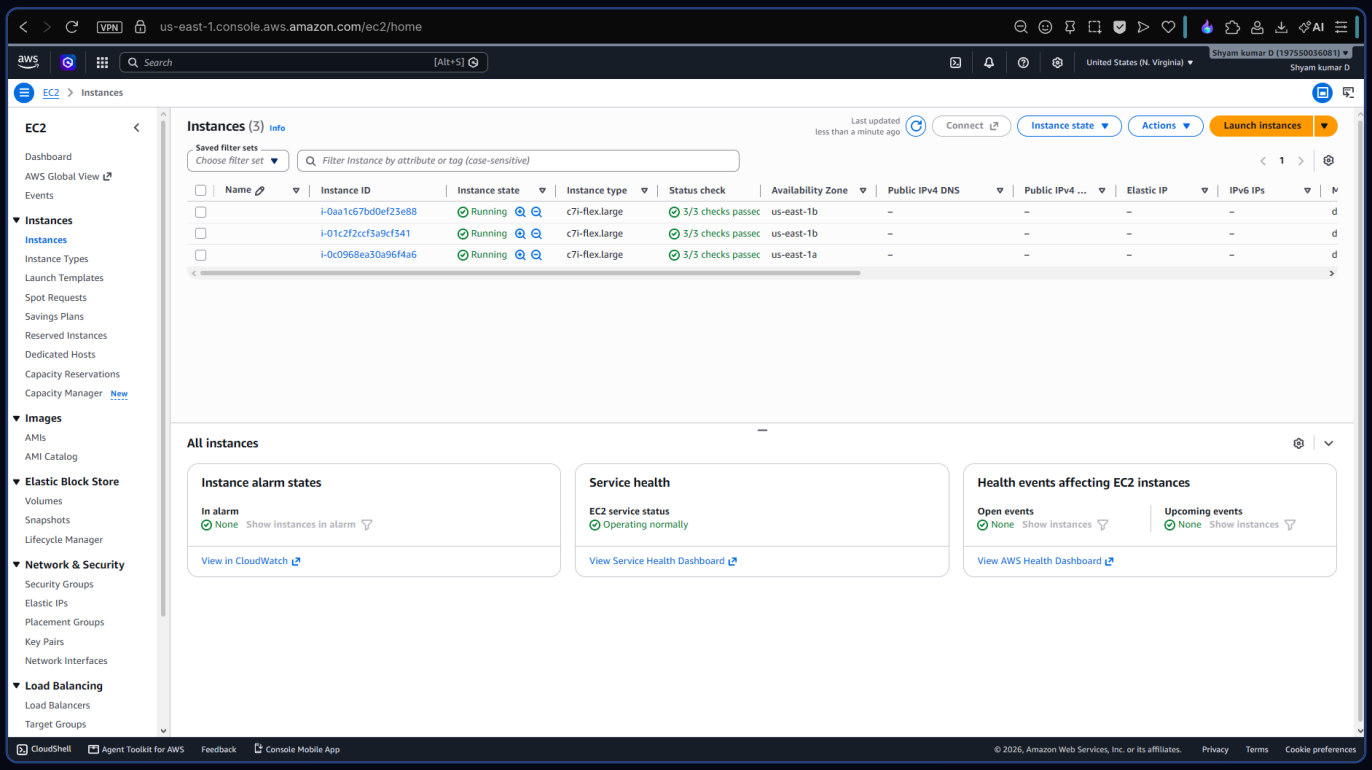
Proof 08 — EKS Compute: Nodes & Node Groups



3 active **c7i-flex.large** worker nodes, all Ready, managed by the **dev-predictive-eks-general-nodes** node group backed by an EC2 Auto Scaling Group.

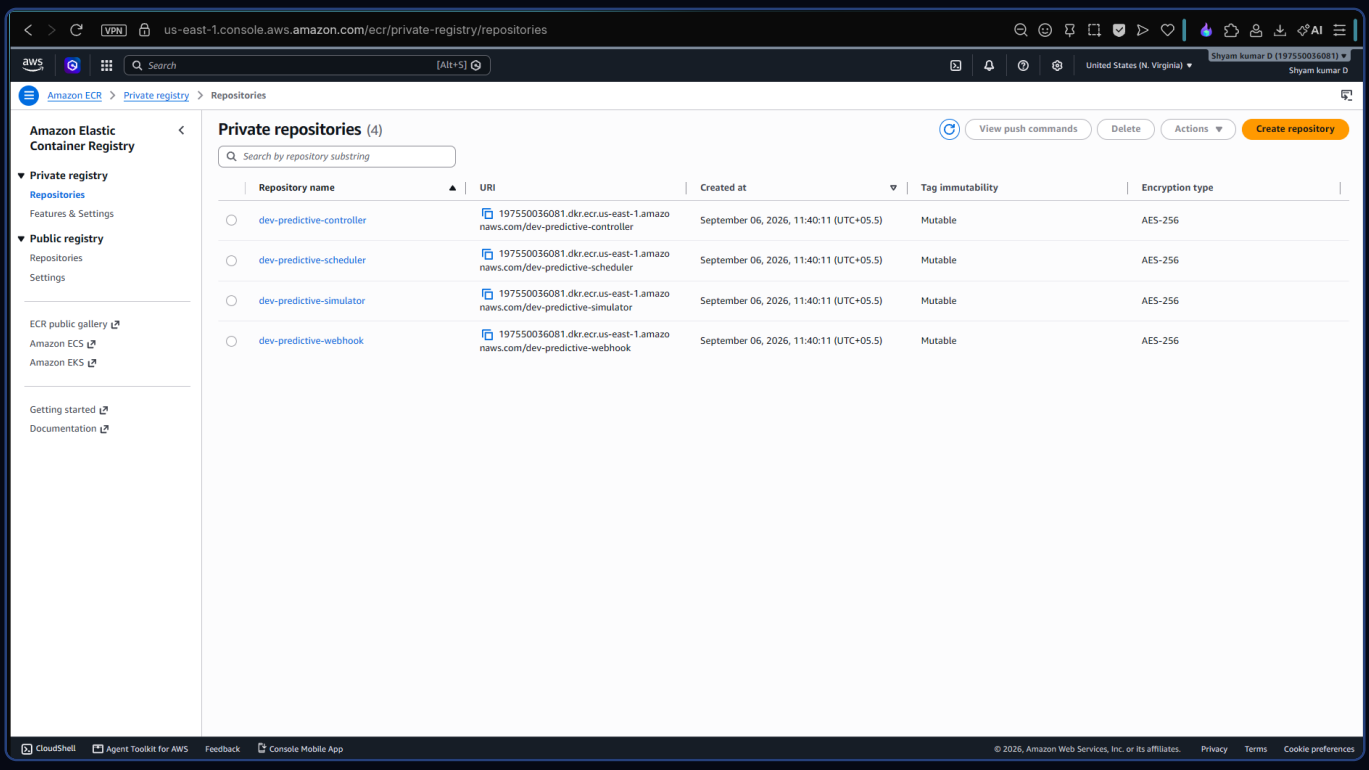
Compute, Registry & Networking Layers

Proof 09 — Amazon EC2: Running Instances



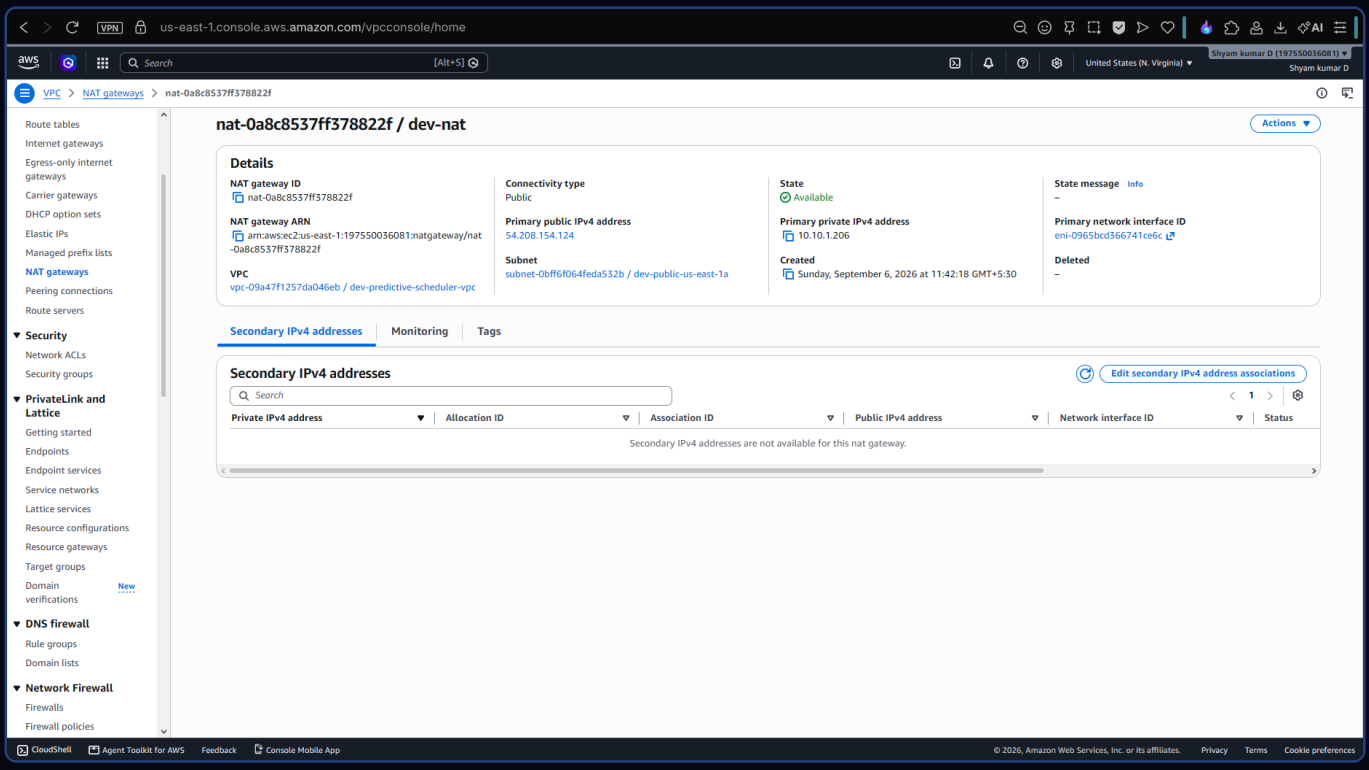
3 running **c7i-flex.large** EC2 instances backing the EKS node group, all passing 3/3 status checks — the exact fleet referenced throughout the soak test.

Proof 10 — Amazon ECR: Private Container Repositories



4 private ECR repositories — **dev-predictive-controller, dev-predictive-scheduler, dev-predictive-simulator, dev-predictive-webhook** — each AES-256 encrypted, confirming the multi-component architecture is fully containerized and deployed.

Proof 11 — Amazon VPC: NAT Gateway Networking



Dedicated **dev-nat** NAT Gateway inside the **dev-predictive-scheduler-vpc**, in the "Available" state — completing the custom, purpose-built VPC networking layer for the cluster.

Conclusion

KubeForecast demonstrates an end-to-end, production-grade approach to solving one of the most under-addressed problems in cloud infrastructure: **structural bin-packing waste caused by naive pod scheduling**. By combining a nanosecond-scale scheduler plugin, a predictive waterline engine, and a fail-safe admission webhook, the project achieves real, AWS-verified cost reductions of up to **66.7%** — with zero disruption to running workloads.

Every result in this report — the scheduling latency, the node consolidation, the AWS billing ledger, and the live REST API calls — was captured directly from a real AWS EKS deployment, not a simulated environment, as evidenced by the console screenshots and dashboard captures above.

Tech Stack Summary

LAYER	TECHNOLOGY
Orchestration	Kubernetes v1.31 (Amazon EKS), Custom Scheduler Framework Plugin
Admission Control	Mutating Admission Webhook (failurePolicy: Ignore)
Compute	AWS EC2 c7i-flex.large (Auto Scaling Node Group)
Networking	Custom VPC, Public Subnet, NAT Gateway, Elastic Load Balancer
Storage & State	S3 State Snapshots (IRSA-authenticated), In-Memory Cache, Prometheus Exporter
Container Registry	Amazon ECR (4 private repositories, AES-256 encrypted)
Delivery	Helm Charts, Multi-stage Docker builds, Terraform IaC
Observability	Live FinOps Dashboard (JARVIS Cloud AI), Cloudflare TLS 1.3 Edge Tunnel

Explore the Full Project

Source code, Helm charts, Terraform modules, and the live video walkthrough are available in the repository below.

github.com/ShyamD2/KubeForecast

Shyam Kumar D · Aspiring Cloud Architect
linkedin.com/in/shyam-kumar-d-951254329 | github.com/ShyamD2